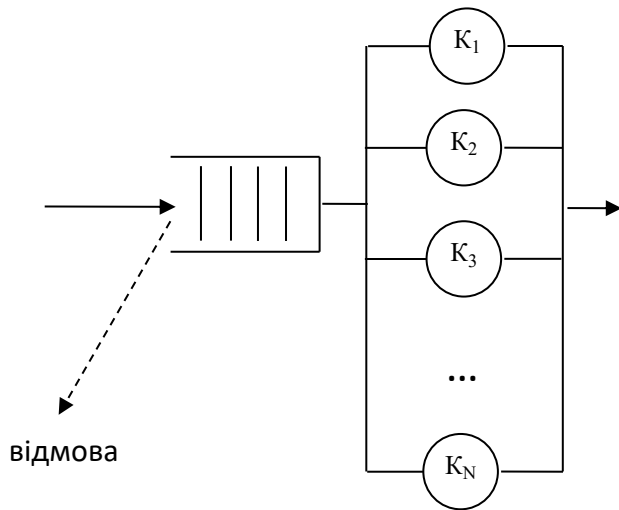


Завдання до комп'ютерного практикуму 5



Правила обслуговування в багатоканальній СМО з обмеженою чергою:

- Об'єкти надходять на обслуговування в СМО через випадкові інтервали часу з заданим середнім значенням (розподілених, наприклад, за рівномірним законом розподілу).
 - Якщо є вільний канал обслуговування, то об'єкт займає його на час обслуговування.
 - В протилежному випадку, об'єкт намагається стати в чергу. Якщо місце в черзі є, то об'єкт займає його. В протилежному випадку, об'єкт залишає СМО необслугованим (збільшується кількість відмов в обслуговуванні).
 - Після завершення обслуговування в каналі об'єкт вважається таким, що завершив обслуговування в СМО (збільшується кількість обслугованих об'єктів).
 - Моделювання здійснюють протягом заданого інтервалу часу (іншим способом є завершення за кількістю об'єктів, які завершили обслуговування). Час моделювання має бути достатньо великим, щоб обслуговування здійснилось для не менш як 1000 об'єктів.
 - За результатом імітації підраховують: ймовірність відмови, середня кількість об'єктів в черзі.
- Для отримання статистично значимої оцінки середньої довжини черги та ймовірності відмови, їх значення оцінюють за кількома прогонами «моделі» (не менше 4)
 - Виведення результатів (для того, щоб отримувати поточну інформацію про перебіг імітації) виконують в окремому потоці через задану часову затримку.

Моделювання паралельних обчислень

Задачі моделювання

Оцінювання ефективності паралельних обчислень застосовується у таких формах:

- 1) Оцінювання прискорення процесу обчислень
- 2) Оцінювання максимально можливого прискорення процесу обчислень

Модель обчислень у вигляді графу « операції-операнди »

Припущення:

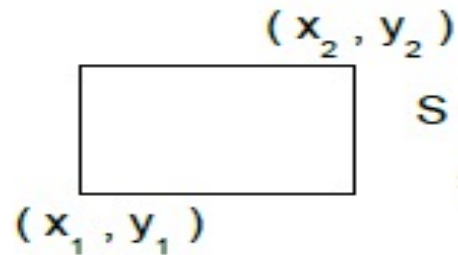
- усі обчислювальні операції виконуються з однаковою з часовою затримкою рівною 1,
- передача даних між обчислювальними пристроями здійснюється миттєво
(!вірно, якщо використовується спільна модель пам'яті в розподіленій системі)

Структура графу

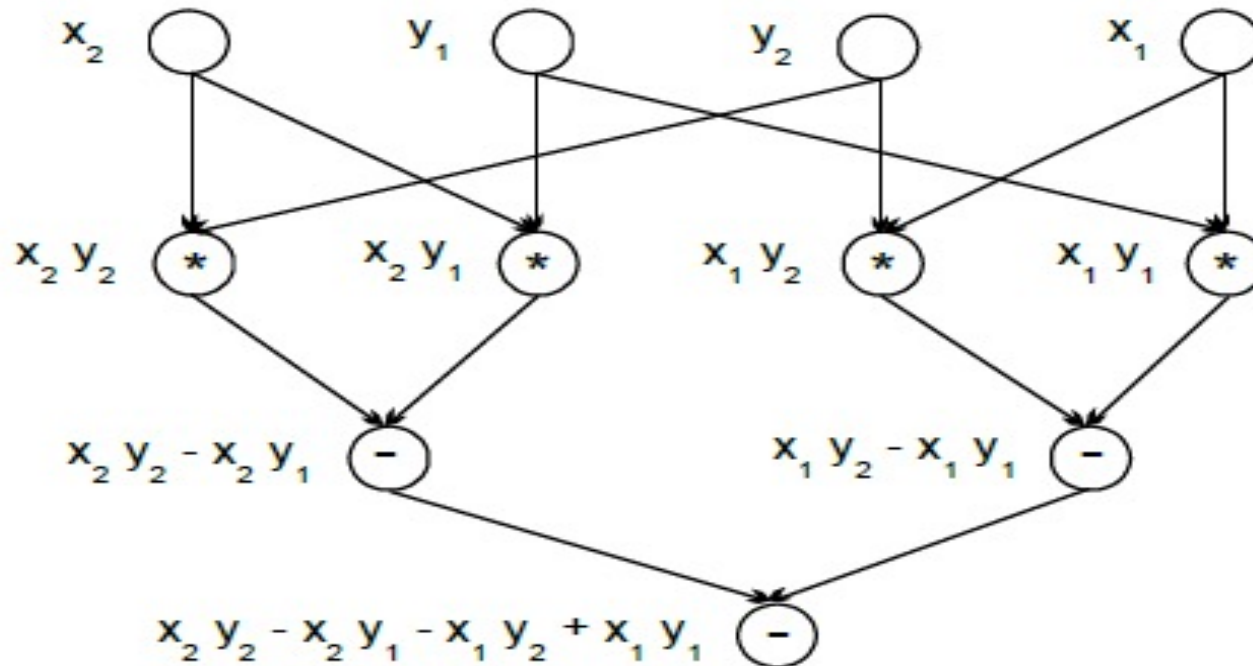
Вершини графу представляють операції алгоритму.

Дуга між вершинами графу існує, якщо операція – кінець дуги використовує результат обчислень операції – старт дуги.

Приклад



$$S = ((x_2 - x_1)(y_2 - y_1)) = \\ = x_2 y_2 - x_2 y_1 - x_1 y_2 + x_1 y_1$$



Визначення часу виконання паралельних обчислень

- Мінімально можливий час виконання паралельного алгоритму визначається довжиною максимального шляху обчислювальної схеми алгоритму:

$$T_{\infty}(G) = d(G),$$

де T_{∞} - час виконання паралельного алгоритму на **паракомп'ютері** (з нескінченно великою кількістю процесорів) $T_{\infty} = \min_p T_p$

- Мінімально можливий час виконання паралельного алгоритму, за умови, що кількість входів у кожну вершину графу не перевищує 2, обмежений знизу величиною

$$T_{\infty}(G) \geq \log_2 n,$$

де n – оцінка складності алгоритму (кількість вхідних даних, наприклад).

Визначення часу виконання паралельних обчислень

- При зменшенні кількості використовуваних процесорів час виконання алгоритму збільшується пропорційно:

$$0 < c < 1 \quad T_{cp} \leq \frac{T_p}{c}$$

Наприклад, при $p=16$, $c=0,5$: $T_8 \leq 2T_{16}$

Окремий випадок: якщо $cp = 1$, то $T_1 \leq pT_p$, або $T_p \geq \frac{T_1}{p}$.

Тобто час виконання алгоритму на p процесорах обмежений знизу величиною $\frac{T_1}{p}$.

- Часу виконання паралельного алгоритму на p процесорах обмежений зверху:

$$\forall p \quad T_p < T_\infty + \frac{T_1}{p},$$

де T_1 - час виконання послідовного алгоритму

Визначення часу виконання паралельних обчислень

- Час виконання алгоритму, який наближений до мінімально можливого часу T_∞ , можна досягнути при такій кількості процесорів:

$$p \sim \frac{T_1}{T_\infty} . \text{ (Дійсно, оскільки } T_\infty \rightarrow \frac{T_1}{p}, \text{ то } p \rightarrow \frac{T_1}{T_\infty})$$

При цьому,

$$\text{якщо } p \geq \frac{T_1}{T_\infty}, \text{ то } T_p \leq 2T_\infty,$$

$$\text{якщо } p < \frac{T_1}{T_\infty}, \text{ то } \frac{T_1}{p} \leq T_p \leq 2 \frac{T_1}{p}$$

Рекомендації до складання паралельних алгоритмів

- 1) При виборі обчислювальної схеми повинен використовуватись граф з мінімально можливим діаметром
- 2) Для паралельного виконання доцільно кількість процесорів визначати величиною $p \sim \frac{T_1}{T_\infty}$
- 3) Час виконання паралельного алгоритму обмежується зверху величинами:

$$\forall p \quad T_p < T_\infty + \frac{T_1}{p}$$

$$\text{if } p \geq \frac{T_1}{T_\infty} \Rightarrow T_p \leq 2T_\infty;$$

$$\text{if } p < \frac{T_1}{T_\infty} \Rightarrow T_p \leq 2 \frac{T_1}{p}$$

Показники ефективності паралельного алгоритму

- **Прискорення**, одержуване при використанні p процесорів для реалізації паралельного алгоритму складності n , оцінюється величиною:

$$S_p(n) = T_1(n)/T_p(n)$$

- **Ефективність** використання процесорів при паралельній реалізації алгоритму оцінюється величиною:

$$E_p(n) = T_1(n)/(p \cdot T_p(n)) = S_p(n)/p$$

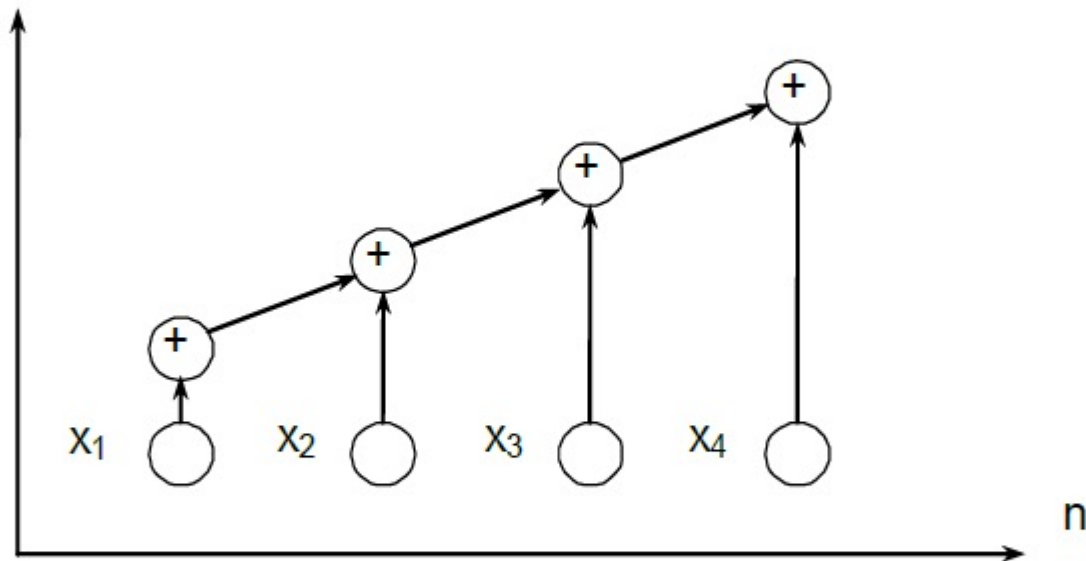
Зауваження: $E_p(n) \leq 1$, $S_p(n) \leq p$. Якщо досягається $S_p(n) > p$, то говорять про суперлінійне прискорення. Це можливо для алгоритмів з нелінійною складністю

- **Вартість** обчислень

$$C_p = pT_p$$

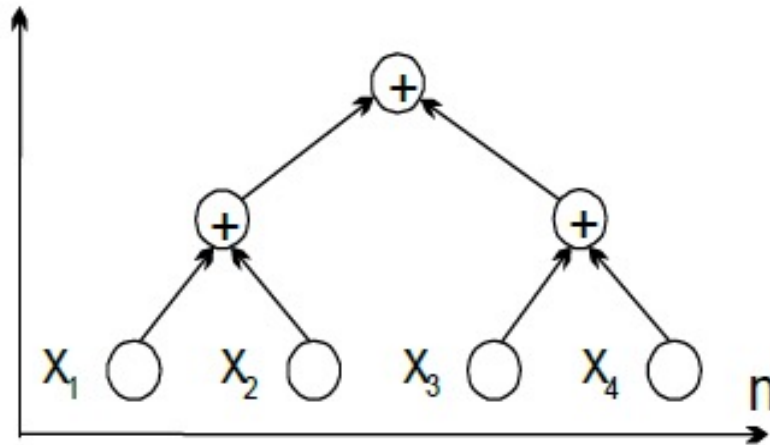
Приклад: алгоритм обчислення суми

Сума $S = \sum_{i=1}^n x_i$ послідовно розраховується у такі кроки: $S=0$, $S=S+x_1$, ... Схеми цих обчислень має такий вид і не може бути розпаралелена:



Приклад: алгоритм обчислення суми

Каскадна схема паралельного алгоритму сумування:



$$T_p = \log_2 n \qquad p = \frac{n}{2}$$

Ефективність алгоритму обчислення суми

Кількість операцій сумування:

$$T_1 = n - 1$$

Кількість паралельних операцій сумування:

$$T_p = \log_2 n$$

Маємо такі оцінки ефективності:

$$S_p = T_1/T_p = (n - 1)/\log_2 n,$$

$$E_p = S_p/p =$$

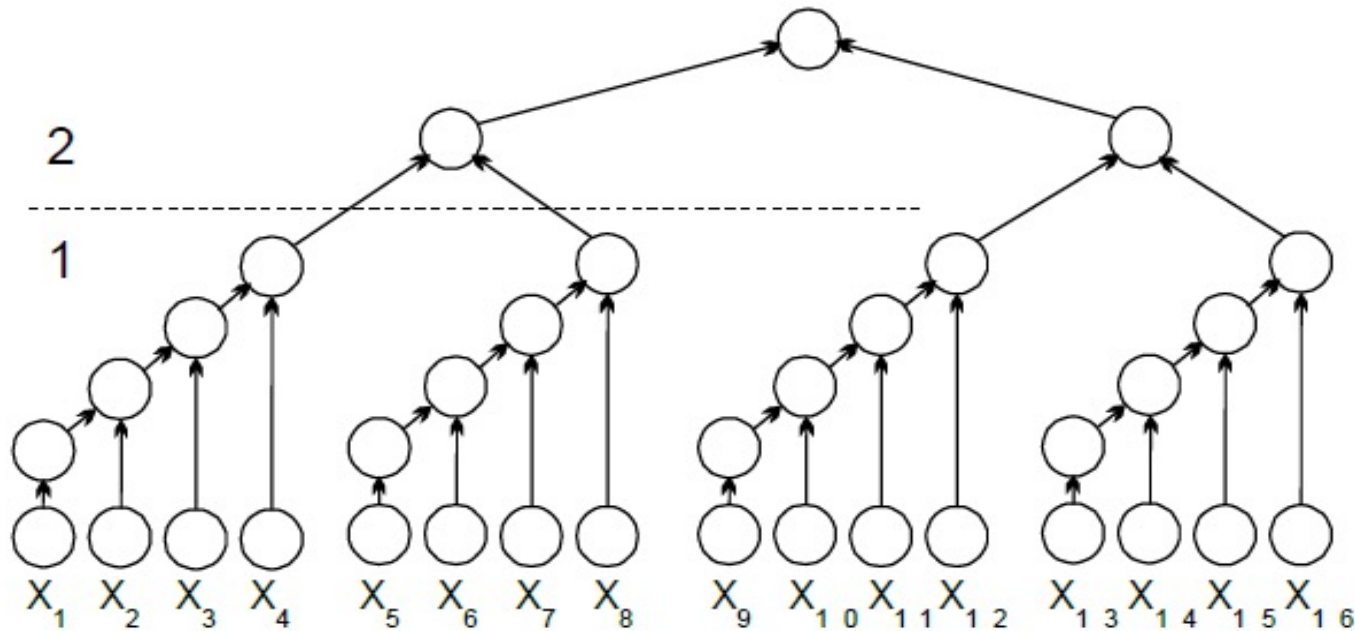
$$(n - 1)/(p \log_2 n) = (n - 1)/\left(\frac{n}{2} \log_2 n\right)$$

При зростанні кількості даних: $\lim_{n \rightarrow \infty} E_p \rightarrow 0$

Модифікована каскадна схема паралельного обчислення сум

Припустимо, що $n = 2^k$, тоді $T_\infty = \log_2 n = k$.

Кількість процесорів (кількість груп) $p = T_1/T_\infty = n/\log_2 n$



$$T_p = k + \log_2 \frac{n}{k} = \log_2 n + \log_2 \frac{n}{k} = 2 \log_2 n - \log_2 k \leq 2 \log_2 n$$

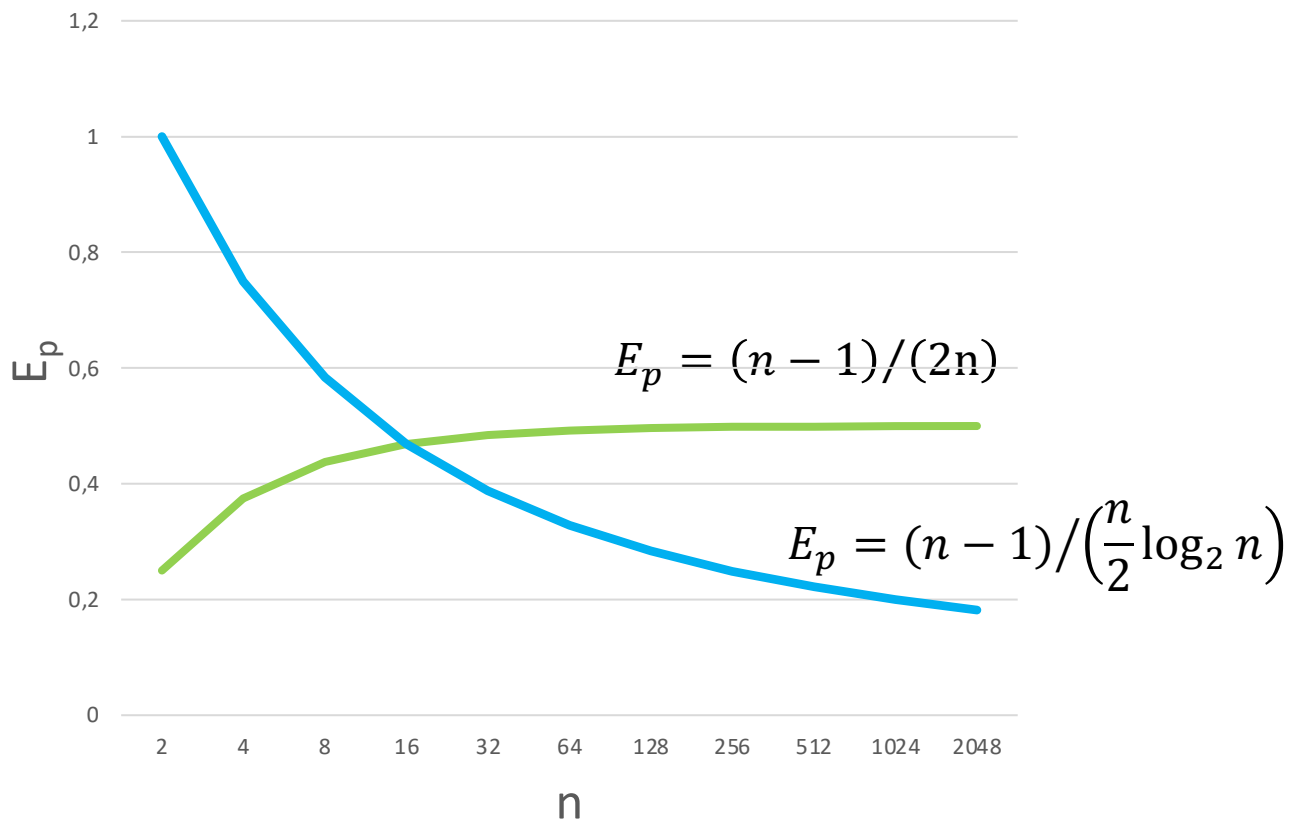
$$T_p \leq 2 \log_2 n, \quad p = \text{кількість груп} = n/k = n/\log_2 n$$

Ефективність модифікованого каскадного сумування

- $S_p = T_1/T_p = (n - 1)/(2\log_2 n)$
- $E_p = T_1/(p \cdot T_p) =$
 $(n - 1)/(n/\log_2 n \cdot 2\log_2 n) = (n - 1)/(2n)$

Маємо (!!!): $\lim_{n \rightarrow \infty} E_p \rightarrow 0,5$

Порівняння ефективності звичайного та модифікованого каскадних алгоритмів



Виведення закону Амдала

f – частка коду, який розпаралелений

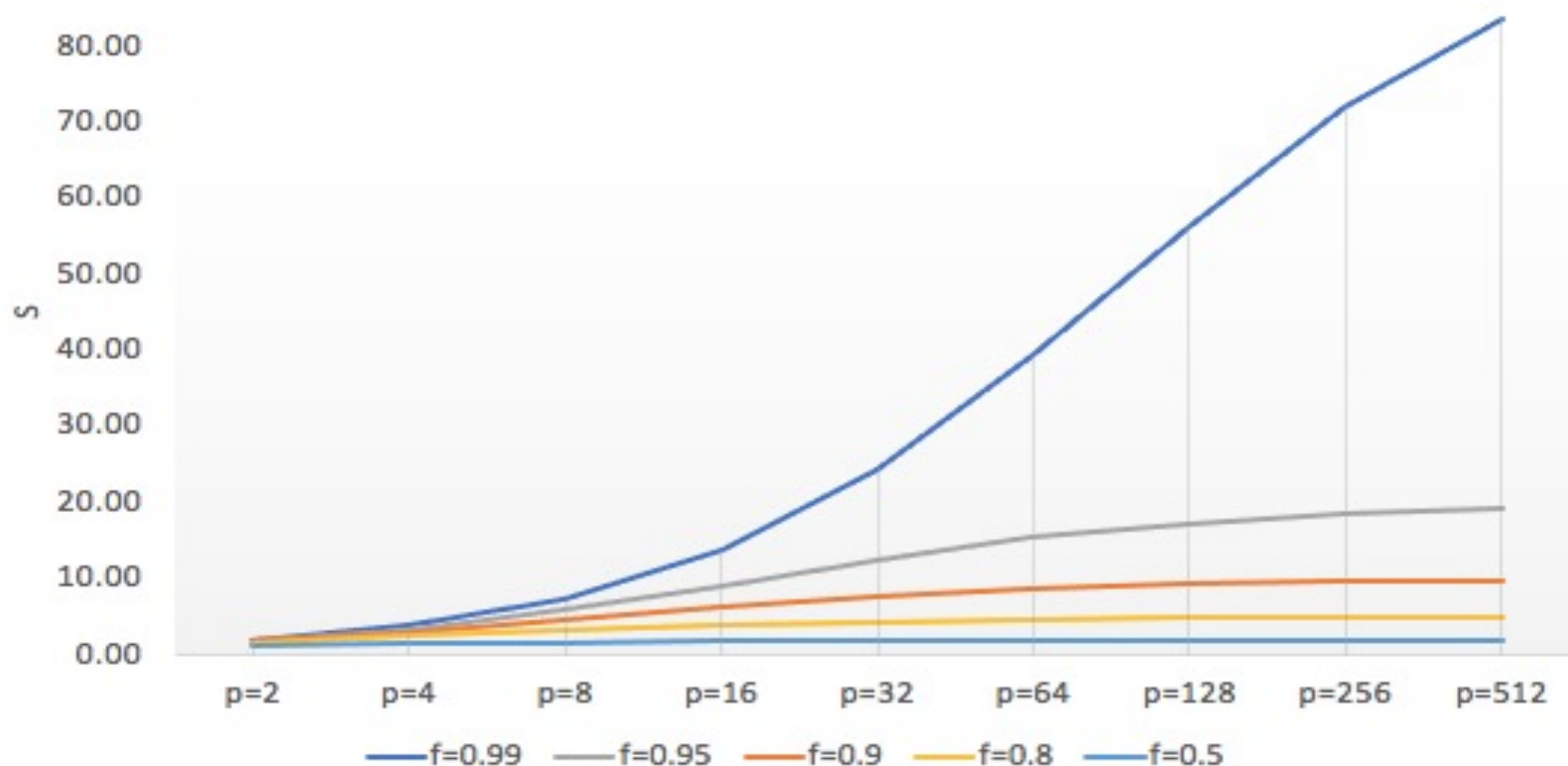
$(1 - f)$ – частка коду, який виконується послідовно

$$S_p = \frac{T_1}{T_p} \leq \frac{(1 - f)T_1 + fT_1}{(1 - f)T_1 + \frac{fT_1}{p}} = \frac{1}{(1 - f) + \frac{f}{p}}$$

Послідовна
частина коду

Паралельна
частина коду

Графічне представлення закону Амдала $S(f,p)$



Залежність прискорення S паралельних обчислень від кількості використовуваних процесів p при збільшенні частки f обчислень, що розпаралелені в програмі

Максимально досяжний паралелізм

- Закон Амдала

$$S_p \leq \frac{1}{h + \frac{f}{p}} \leq \frac{1}{h},$$

де $h = 1 - f$ – частка послідовних обчислень в алгоритмі.

Наприклад, для каскадної схеми $h = \frac{k}{n} = \frac{\log_2 n}{n}$, тому прискорення обмежене величиною

$$S_p \leq \frac{1}{h} = n / \log_2 n$$

- Ефект Амдала:** оскільки $h(n)$ спадає при зростанні n , то $S_p(n)$ є зростаючою функцією.

Аналіз масштабованості паралельних обчислень

- Паралельний алгоритм називають **масштабованим** (scalable), якщо при зростанні кількості процесорів він забезпечує збільшення прискорення при збереженні постійного рівня ефективності використання процесорів.
- Накладні витрати, які мають місце при виконанні паралельного алгоритму, оцінюються величиною

$$T_0 = pT_p - T_1$$

Аналіз масштабованості паралельних обчислень

- Для бажаного значення ефективності E виконано співвідношення:

$$\frac{T_0}{T_1} = \frac{1 - E}{E} = \frac{1}{K} \Rightarrow T_1 = KT_0, \text{ де } K = \frac{E}{1 - E}$$

- З останнього співвідношення може бути одержана залежність виду $n = F(p)$ між складністю задачі та кількістю процесорів. Цю залежність називають функцією **ізоефективності**.

Наприклад, для алгоритму обчислення суми маємо:

$$T_0 = pT_p - T_1 = p \left(\frac{n}{p} + \log_2 p \right) - n = p \log_2 p$$

$$T_1 = KT_0 \Rightarrow n = Kp \log_2 p$$

Наприклад, при $E = 0,5$ (тобто $K = 1$) та $p = 16$ з останньої формули отримуємо $n = 64$.